tier1_full_lifecycle_e2e

tier1_full_lifecycle_e2e.sh — Tier-1 emulated-device FULL OTA-LIFECYCLE E2E on podman

Last verified: 2026-06-10 (run-id `20260610T111918Z-full-lifecycle`, podman 5.8.2, arm64 Linux guest)

Overview

`tests/emulator/tier1_full_lifecycle_e2e.sh` closes the **honest gap** the prior Tier-1 e2e (`tier1_container_e2e.sh`) noted: that harness only exercised the clean on-target **204** path because **no deployment was staged**. This harness drives the **COMPLETE OTA round-trip** against a containerized Helix OTA control plane, with no live hardware:

1. boots the control plane (`ota-server`) in a podman pod configured with a real **EPHEMERAL ed25519 trust key** (`HELIX_ARTIFACT_PUBKEY`) so signed artifact uploads verify;
2. via the admin REST API **stages a real deployment** at a version **higher than the device's current**: mint key → sign + upload artifact (201, `verified=true`) → create release → create an all-targets deployment for the device's model/os;
3. runs the `ota-device-emu` container (the `containers/` submodule runtime image, §11.4.76) on `current-version` **below** the release;
4. **asserts** the device gets a **200 update offer carrying the `deployment_id`**, reports the **full telemetry lifecycle ACCEPTED** (`rejected=0`), advances its version, and that a **second run returns 204 on-target**;
5. **cross-checks the server-side telemetry history** (`GET /devices/{id}/telemetry`) shows the `download_started → installing → installed → verifying → success` lifecycle, every event stamped with the staged `deployment_id`.

Anti-bluff (§11.4 / §11.4.107): the proof is the **full lifecycle**, not a single 204. Every PASS is backed by real captured podman logs + the emulator Outcome JSON + the server telemetry history under `docs/qa/<run-id>-full-lifecycle/`. A failed step reports the real error and exits non-zero — it never fakes green.

Prerequisites

- **podman** with a running machine (`podman machine start`). The Linux guest is **arm64**, so binaries cross-compile `GOOS=linux GOARCH=arm64`.
- **Go** toolchain on the host. The server module (`server/go.mod`) has `replace` directives at sibling paths outside `server/` (the §11.4.28 co-developed bricks), so the binaries build **on the host** where those resolve — not in a container.
- Host signing tools (the artifact-staging contract mirrors `tests/e2e/pipeline_signed.sh`): `openssl` (≥3, ed25519), `xxd`, `base64`, `curl`, `jq`, `python3`. The ed25519 keypair is **ephemeral**: generated into a `mktemp` dir that is `rm -rf`'d on exit, never committed (§11.4.10).

Usage

```
bash tests/emulator/tier1_full_lifecycle_e2e.sh
KEEP_UP=1 bash tests/emulator/tier1_full_lifecycle_e2e.sh   # leave the stack up to inspect
```

Optional environment overrides: `PODMAN`, `CP_IMAGE`, `EMU_IMAGE`, `ADMIN_USER`, `ADMIN_PASS`, `HELIX_PORT`.

What it does (phases)

- **KEYS (0)** — mint the ephemeral ed25519 artifact-trust keypair; the raw 32-byte public key is base64-configured into the control plane as `HELIX_ARTIFACT_PUBKEY`.
- **BUILD (a/b)** — cross-compile static `linux/arm64` `ota-server` + `ota-device-emu` on the host; `podman build` the control-plane image, the `containers/`-submodule base emulator image, and a derived image that `COPY`s the binary in (the macOS podman machine does not mount the repo path, so binaries are baked in, not volume-mounted).
- **BOOT (c) + HEALTH (d)** — create a pod publishing `HELIX_PORT`, start the control plane with the trust key, then gate on **both** `/healthz` (200) **and** `/api/v1/auth/login` answering (not a 404 from a stale/wrong server). A **stale-listener guard** fails fast if `HELIX_PORT` is already serving.
- **STAGE (e)** — operator login → sign + upload artifact (assert 201 + `verified=true`) → create release → create all-targets deployment (capture `deployment_id`).
- **RUN 1 (f) + ASSERT (g)** — run the emulator on `current-version` below the release; assert from the Outcome JSON: register (`device_id`), `on_target=false`, `offered_version` == release, `deployment_id` == staged, `applied=true`, `from→to` version advance, `telemetry_accepted≥1` `telemetry_rejected=0`, `healthy=true`.
- **SERVER CROSS-CHECK (h)** — `GET /devices/{id}/telemetry`: assert each of `download_started / installing / installed / verifying / success` is present and the `success` event carries the staged `deployment_id`.
- **RUN 2 (i)** — same device on the new version re-checks: assert `on_target=true`, `applied=false` (clean 204 on-target).

Outputs / evidence

Written under `docs/qa/<run-id>-full-lifecycle/`:

- `tier1_full_lifecycle_transcript.txt` — full timestamped run transcript.
- `emulator_run1_outcome.json` / `emulator_run2_outcome.json` — the emulator's structured Outcome (device_id, applied, from/to version, deployment_id, telemetry counts, healthy).
- `server_telemetry_history.json` — the server-side lifecycle history.
- `control_plane_boot.log` / `control_plane_after_run1.log` — control-plane logs.

Edge cases & honest notes

- **`target_count=0` at deploy time is expected and benign.** The all-targets deployment is created **before** the device registers, so it matches 0 currently-registered targets; the `client/update` resolution is **release-based** for the device's model/os, and the active deployment's id is attached to the offer regardless. The emulator self-serves that id (`otaprotocol.UpdateAvailable.DeploymentID`) and the success lifecycle is ACCEPTED end-to-end (proven by the server history cross-check).
- **Stale listener.** A prior run's pod (or any process) still serving on `HELIX_PORT` would make the published port silently bind elsewhere and a later request hit the wrong server (`/healthz` 200 but `/api/v1/...` 404). The stale-listener guard + the login-answers health gate catch this and fail with a clear message; free the port or set `HELIX_PORT` to a free one.
- **Cleanup** runs on every exit path (`trap … EXIT INT TERM`, §11.4.14): the pod is removed and the ephemeral key dir + build-staging files are deleted (unless `KEEP_UP`).
- **Flashing is the only emulated act** — login, register, update-check, apply, and telemetry are all real HTTP calls against the real control plane.

Related scripts

- `tests/emulator/tier1_container_e2e.sh` — the on-target-204 Tier-1 round-trip (the harness this one extends to the full lifecycle).
- `tests/emulator/tier1_fleet_e2e.sh` — the multi-device fleet variant.
- `tests/e2e/pipeline_signed.sh` — the signed artifact-pipeline contract this harness's staging mirrors.